

1. SELECT и конкатенация строк.

```
select FirstName + ' ' + LastName as FullName  
from dbo.Students1
```

Что происходит:

+ в SQL Server используется для объединения строк.

' ' – пробел между именем и фамилией.

as FullName – псевдоним (alias) столбца.

dbo – схема (контейнер объектов в базе).

Важно:

если одно из полей NULL, результат будет NULL.

Пример проблемы:

```
FirstName = NULL  
LastName = 'Иванов'
```

Результат → NULL

Чтобы этого избежать:

```
select ISNULL(FirstName, ' ') + ' ' + ISNULL(LastName, ' ')  
from Students1
```

2. CAST() и CONVERT()

```
cast(Grants as nvarchar(10))  
convert(nvarchar(10), Grants)
```

Что делают?

Преобразуют типы данных.

Например:

int -> nvarchar

datetime -> varchar

float -> int

В чём разница между CAST и CONVERT?

CAST - ANSI стандарт / проще

CONVERT – специфичен для SQL-серверов, позволяет форматировать даты

Пример форматирования даты:

```
convert(varchar, BirthDate, 104)
```

104 – формат dd.mm.yyyy

3. TOP

```
SELECT TOP 20 PERCENT ...
```

Что делает?

Возвращает первые 20% строк.

Важно:

Без ORDER BY порядок строк не гарантирован.

Правильный вариант:

```
SELECT TOP 5 *  
FROM Students1  
ORDER BY BirthDate DESC;
```

4. DISTINCT

```
SELECT DISTINCT FirstName  
FROM Students1;
```

Что делает?

Удаляет дубликаты из результата.

Как работает?

SQL сначала формирует выборку, затем убирает повторяющиеся строки.

Если указать несколько колонок:

```
SELECT DISTINCT FirstName, LastName
```

Уникальность будет проверяться по комбинации столбцов.

5. WHERE

```
WHERE id > 3
```

Что делает?

Фильтрует строки **до** вывода.

Порядок выполнения запроса:

1. FROM
2. WHERE
3. SELECT
4. ORDER BY

6. LEN()

```
WHERE LEN(Grants) > 4;
```

Что делает?

Возвращает длину строки.

Если Grants – int, SQL выполнит неявное преобразование.

Лучше явно:

```
LEN(CAST(Grants AS varchar))
```

Особенность:

LEN() не считает пробелы в конце строки.

7. AND / OR

```
WHERE MONTH(BirthDate) >= 9  
AND MONTH(BirthDate) <= 11;
```

AND

Оба условия должны быть TRUE.

OR

Хотя бы одно TRUE.

При смешивании:

```
WHERE A = 1 OR B = 2 AND C = 3
```

Сначала выполнится AND.

Правильно:

```
WHERE (A = 1 OR B = 2)
```

```
AND C = 3
```

8. Работа с датами

```
YEAR(BirthDate) % 2 = 0
```

Что делает?

Проверяет чётность года.

% - остаток от деления.

Пример:

```
5 % 2 = 1
```

```
4 % 2 = 0
```

9. Проверка на NULL

```
WHERE Grants IS NOT NULL;
```

Важно:

NULL – это не значение, а отсутствие значения.

Нельзя писать:

```
WHERE Grans = NULL [ X ]
```

Правильно:

```
IS NULL
```

```
IS NOT NULL
```

10. ORDER BY

```
ORDER BY BirthDate DESC, FirstName ASC;
```

Как работает:

1. Сортирует по дате (убывание).
2. Если даты равны – сортирует по имени.

11. IN

```
WHERE LastName IN ('Иванов', 'Петров')
```

Эквивалент:

```
WHERE LastName = 'Иванов'  
OR LastName = 'Петров'
```

Работает быстрее и чище визуально.

12. BETWEEN

```
WHERE BirthDate BETWEEN '1999-01-01'  
AND '2002-01-01'
```

Особенность:

BETWEEN включает границы.

Эквивалент:

```
>= '1999-01-01'  
AND <= '2002-01-01'
```

Для datetime может захватить не весь день.

Лучше писать:

```
>= '1999-01-01'  
AND < '2002-01-02'
```

13. LIKE

```
LIKE 'П%'
```

Шаблоны:

% - любое количество символов

_ - один символ

[abc] – один из

[^abc] – кроме

Пример:

```
LIKE '_ванов'
```

Найдёт:

- Иванов
- Пванов

14. INSERT

```
INSERT INTO Students1(id, BirthDate, FirstName, LastName)  
VALUES (...)
```

Как работает?

Добавляет строку.

Если поле NOT NULL – его обязательно указывать.

15. UPDATE

```
UPDATE Students1  
SET Grants += 500  
WHERE Grants IS NOT NULL;
```

Важно:

Без WHERE обновятся все строки.

16. DELETE

```
DELETE FROM Students1  
WHERE id > 9;
```

Удаляет строки.

Без WHERE – удалит всё.

17. CREATE TABLE

```
Id Int PRIMARY KEY
```

PRIMARY KEY:

- Уникальность
- Автоматически создаёт индекс
- Не может быть NULL

18. CHECK

Что такое CHECK?

Это ограничение (constraint), которое проверяет условие перед вставкой (INSERT) или изменением (UPDATE) данных.

Если условие возвращает TRUE – запись сохраняется.

Если FALSE – SQL выдаёт ошибку.

Пример:

```
GRANTS int check(GRANTS IN (1,2,3,4,5))
```

Что это означает:

- В колонку GRANS можно записать только 1, 2, 3, 4, или 5.
- Если попытаться вставить 6 – будет ошибка.
- Если вставить 3 – всё нормально.

Как работает внутри?

SQL при каждой вставке или обновлении проверяет условие CHECK:

Если TRUE – сохранить.

Если FALSE – ошибка.

Проверка идёт на уровне базы данных, а не приложения.

То есть даже если кто-то подключится напрямую к базе – правило всё равно сработает.

Разные способы создания CHECK.

На уровне столбца.

```
Grants int CHECK (Grants > 0)
```

*Используется, когда условие касается только **одного** столбца.*

На уровне таблицы.

```
CREATE TABLE Students1
(
    Id int,
    Grants int,
    BirthDate date,
    CHECK (Grants >= 0 AND BirthDate > '1900-01-01')
);
```

*Используется, когда условие зависит от **нескольких** столбцов.*

CHECK и NULL – важный момент!

CHECK не запрещает NULL. Почему?

Потому что:

- Если значение NULL
- Выражение возвращает UNKNOWN
- SQL пропускает строку

Пример:

```
Grants int CHECK (Grants > 0)
```

Можно вставить:

```
Grans = NULL
```

Если нужно запретить:

```
Grants int NOT NULL CHECK (Grants > 0)
```

Более сложные примеры.

Проверка диапазона.

```
CHECK (Grants BETWEEN 0 AND 50000)
```

Проверка даты.

```
CHECK (BirthDate <= GETDATE())
```

Запрещает дату из будущего.

Проверка зависимости столбцов.

```
CHECK (EndDate >= StartDate)
```

Очень частый реальный сценарий.

Добавление CHECK к существующей таблице.

```
ALTER TABLE Students1
```

```
ADD CONSTRAINT CHK_Grants_Positive  
CHECK (Grants >= 0);
```

Зачем имя ограничения?

- Можно удалить позже
- Можно управлять им

Удаление:

```
ALTER TABLE Students1  
DROP CONSTRAINT CHK_Grants_Positive;
```

Что нельзя использовать в CHECK?

- Подзапросы
- SELECT
- Другие таблицы
- Нестабильные функции

Это логическое ограничение строки, а не бизнес-логика приложения.

Когда использовать CHECK?

- Для проверки диапазона чисел
- Для проверки формата
- Для ограничения набора значений
- Для контроля логики между столбцами

Разница между CHECK и другими ограничениями.

Ограничение:

Для чего:

PRIMARY KEY

Уникальность + NOT NULL

UNIQUE	Уникальность
NOT NULL	Запрет пустых значений
DEFAULT	Значение по умолчанию
CHECK	Логическое условие

Практический пример более реалистичной таблицы, которая защищена от многих ошибок:

```
CREATE TABLE Employees
(
    Id int PRIMARY KEY,
    Salary int NOT NULL CHECK (Salary > 0),
    Age int CHECK (Age BETWEEN 18 AND 65),
    HireDate date CHECK (HireDate <= GETDATE())
);
```

Для чего реально применяется CHECK?

- Перенести бизнес-логику в базу.
- Защитить данные от мусора.
- Сделать систему устойчивее.
- Снизить зависимость от кода приложения.